DESIGN

Email template builder

A merchant edits the shop's emails by dragging, not by editing ERB

Status	**Shape agreed, nothing built**
Spree	**5.6.1 Community Edition**
Updated	**22 September 2026**

Everything marked [probed] was checked against a running system, not reasoned about.

A merchant edits the shop's emails by dragging, not by editing ERB.

> Nothing here is built yet. This is the shape, agreed before code.
>
> **[probed]** means checked against a running system: Rails 8.1.3 driven with a real resolver, or BuilderJS source read at `~/apps/builderjs`. **[proposed]** means reasoned from code that was read but not yet run. Everything else is a decision. `RESEARCH.md` holds the measurements; the probes are in the session scratchpad and are re-runnable.

---

0. Vocabulary

Eight terms the rest of this document uses as if they were shared.

BuilderJS — a third-party drag-and-drop page builder, at `~/apps/builderjs`. It runs in the browser, inside an iframe it calls the canvas. It is built with `npm run build` and its `dist` is served by us.

Page tree / builder tree / element tree — the same object at different moments: the JSON document BuilderJS edits. One name would have been better.

Marker — a placeholder the merchant drags in, exported as a bare tag such as `<div data-spree-block="order_summary">`. It is where Spree's own content goes.

Block — what a marker resolves to: one of Spree's existing partials, with its locals bound by us.

View resolver — a Rails object (`ActionView::Resolver` subclass) that answers "is there a template called X" with `ActionView::Template` objects. **Prepended** here means `prepend_view_path`, not Ruby's `Module#prepend`.

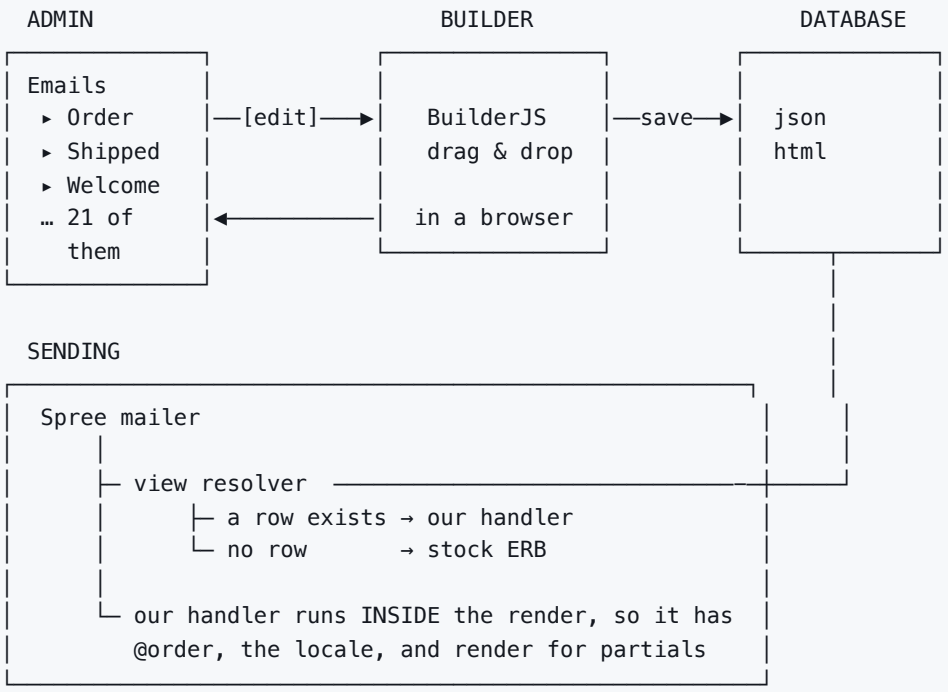
Template handler — the thing that turns a template's source into the Ruby that renders it. ERB is one. We add our own.

Shop / estate — 7 shops, each its own database, its own container and its own brand, keyed by `ENV['SHOP_KEY']`.

`emails:smoke` — `lib/tasks/email_smoke.rake`. Builds every transactional email on a shop and checks the rendered body. It is the gate that would notice a broken template before a customer does.

---

1. The whole thing in one picture



Three properties matter more than any feature:

Fallback is free. No saved row means Rails renders the stock ERB exactly as today. There is nothing to switch on, and "put it back" is a delete.

Per shop. 7 shops, 7 brands. Getting this wrong is how a payment-link email once carried another company's bank details.

Spree keeps the money. A builder exports static HTML, and six of these emails print line items, prices and taxes. The author places *where* the order summary goes; Spree decides *what is in it*.

---

2. Where things convert

Four boundaries. Every hard decision sits on one of them.

```
①  drag  ──▶  JSON          in the browser, by BuilderJS
②  JSON  ──▶  HTML          in the browser. There is no server path  [probed]
③  HTML  ──▶  final email   during the render, by our handler
④  send  ──▶  Spree         a view resolver, prepended  [probed]
```

① drag → JSON

BuilderJS holds a page tree. Email is a *downgrade of a page*, not a template type of its own, so authors build modern HTML and the serializer does the email conversion. Hand-writing `<table>` markup fights the pipeline and is explicitly the wrong instinct.

② JSON → HTML, and only in a browser [probed]

`Builder.js:1696` clones `this.iframe.contentDocument.documentElement`. There is no headless renderer anywhere in BuilderJS; its own tooling drives a real Chromium through Playwright.

Two consequences, and they are the ones to remember:

- **The stored HTML is the only copy of the rendered form.** The server cannot regenerate it from the JSON.
- So a change on the Spree side, a renamed partial for instance, **cannot be back-filled by a migration**. It is a headless-browser job, or reopening each template. 21 emails × 7 shops is 147 documents.

`EmailExportPipeline.prepareForEmail` looks server-runnable and is a trap three ways over [probed]:

- It returns its input **untransformed when there is no DOM**, with no error.
- That check, `HAS_NATIVE_DOMPARSER`, is evaluated at **module load**, so requiring the file before installing jsdom globals poisons it permanently.
- There is a **second** silent pass-through above it: a wrong `artefactType` option returns the input unchanged too.
- Its header comment promises a cheerio hook for Node callers. **That hook does not exist.** The only way in is defining global `DOMParser` and `document`.

And `scripts/core/transform-email-samples.cjs`, which says it mirrors the pipeline "byte-for-byte", is a **fork that has drifted**: 8 steps where the browser pipeline runs 15. Any plan that says "run the pipeline on the server" is choosing between a function that silently does nothing and a fork that is already behind.

③ HTML → the email

Where this runs is the load-bearing question, and the first draft of this document did not answer it. The swap needs `@order`, the store, the locale and `render`. Three obvious homes are all ruled out by things already measured:

	why not
an ActionMailer interceptor	`@order` is gone by then, and `with_store_locale` has already exited, so `Spree.t` and money fall back to the default locale. This repo has a written lesson about exactly one missing locale switch in a mailer path
inside the template source	the source then differs per email, which makes the Template uncacheable and runs into the memory problem below
a decorator per mailer	the pattern `CLAUDE.md` ranks last, and one file for each of 21 emails

So: **a custom template handler** [proposed]. A handler receives `(template, source)` and returns Ruby source. Ours returns a fixed line:

```
Spree::EmailTemplates.swap(<row id>, self)
```

About forty bytes, whatever the size of the email. `self` is the live view context, so assigns, helpers, `render`, routes, currency and `I18n.locale` are all present and still inside `with_store_locale`.

The merchant's HTML never enters compiled Ruby at all. That is a stronger property than the first draft claimed, and the first draft's claim was false: Rails compiles every template, and even the stock `Raw` handler produces `"#{source.inspect}.html_safe;"`, so "nothing is compiled" was never true. What is true is that nothing of the *merchant's* is.

Inside `swap`, the stored HTML is walked and two kinds of marker replaced:

```
<div data-spree-block="order_summary">  →  the RENDERED output of Spree's own
                                           partial, locals bound in Ruby
{{ order.number }}                      →  looked up in a whitelist map
everything else                         →  left exactly as it is
```

The template cache, which is not optional [probed]

Measured in `actionview-8.1.3`, and this changes the design rather than decorating it:

- `Template#compile!` does `module_eval` into `view.compiled_method_container`.
- `Template#method_name` includes `__id__`, so **a new Template object means a new method**.
- `view_context_class` is a **process-global memo**, cleared only on reload, which is development.
- The base `ActionView::Resolver` has **no cache**. Only `FileSystemResolver` does.

A resolver that builds a fresh Template per lookup therefore defines one permanently retained method **per email sent**, on containers that idle at about 420MB against a 768MB limit and already have an OOM history here.

So the resolver memoises Templates on `(shop_key, view_path, locale, updated_at)`. Saving changes `updated_at`, which is the invalidation.

④ Spree → the saved template

A **prepended** view resolver on `Spree::BaseMailer`. Not appended [probed]: `PathSet#find_all` returns on the first resolver that answers, and **all 21 emails already have a stock ERB view in a path that precedes any appended one**. Appended, the resolver is never reached at all, on any shop — the merchant presses Save to no effect, with no error and no log. One shop having 7 designs of its own on disk makes it worse, but is not the cause.

Prepended, a database row silently overrides a file a developer committed. So **which wins is a decision, written here and logged at boot**, not an accident of initializer sort order. It is: **a saved template wins.**

---

3. What holds what

One table. Spree ships none of its own, so this is additive and reversible.

```
spree_email_templates
  shop_key    one key per shop, from ENV.  NOT store_id        [probed]
  view_path   'spree/order_mailer/confirm_email'
  locale      'en', 'en-CA', 'en-business'   (per-shop locale files, not browser locales)
  json        the builder tree.  SOURCE OF TRUTH
  html        derived. Produced in a browser, never on the server
  subject     nullable. Subjects are built in Ruby, not in views
  updated_at  the cache key above depends on this
  unique (shop_key, view_path, locale)
```

Why `shop_key` and not `store_id` [probed]. A resolver is handed `(name, prefix, partial, details, key, locals)` : no mailer, no store. The only global available is `Spree::Current.store` , which falls back to the default store without ever raising. This estate is one database per shop, so it is right today by luck and silently wrong the day any database holds two stores.

Where the plain-text part comes from. BuilderJS produces no text output of any kind, so there is nothing to store. The resolver returns **two** templates: the saved HTML, and the shop's **stock `.text.erb` read off disk**, untouched.

It has to return both [probed]: `ActionMailer::Base#each_template` calls `find_all` **once for all formats**, and the path set returns on the first resolver that answers, so html and text must come from the *same* resolver. A resolver serving html only produces a **single-part** message, the stock text part is never looked for, and `emails:smoke` prints `text 0 B` on an **ok** line and exits 0. Green, while every email on every shop has lost its plain-text alternative.

The cost of this choice, stated rather than discovered: the text part keeps the shop's original wording and does not follow the merchant's edits.

---

4. Blocks are per email, not per name

The first draft wrote `render 'spree/shared/order_summary'` . **That partial does not exist** [probed]. The real one is `_order_summary_section` , called from four views with *different* locals:

```
confirm_email:  line_items: @order.line_items,               order: @order
shipped_email:  line_items: @shipment.manifest.map(&:line_item),
                order: @shipment.order,  totals: false
```

and the partial's own default is `totals: local_assigns.fetch(:totals, true)` .

So a merchant dragging "Order summary" into the shipped email gets `totals: true` and prints **the whole order's total beside a partial list of items**. Wrong money, in front of a customer, produced by a drag.

The blocks therefore live in a registry in our own code, keyed by email, and the palette only offers what that email supports:

```
{ 'spree/order_mailer/confirm_email' => {
    'order_summary' => ->(a) { ['spree/shared/order_summary_section',
                               line_items: a[:order].line_items, order: a[:order]] } } }
```

`a` is the view context's assigns, which the handler in §2③ is what makes available.

The marker must originate in the builder's element tree, never be post-processed into the exported HTML. BuilderJS's cycle is `json → canvas → content`, and on save it writes back whatever the element tree holds, so a placeholder that exists only in the HTML is destroyed by the merchant's first save, silently. The Acelle integration, the other product in this office built on BuilderJS, hit exactly this.

The element is a supported extension [probed]: `Builder.registerElement` before `load()`, `widgetsBox.addWidget` inside the load callback, and `beforeExport(outputMode)` to draw a readable preview on the canvas and a bare marker on export. Twelve stock elements already do this.

---

5. Where the code goes

```
app/models/spree/email_template.rb          the row
lib/spree/email_template_resolver.rb        the seam. Serves html AND text, memoised
lib/spree/email_template_handler.rb         returns the fixed call. Nothing of theirs compiled
lib/spree/email_templates.rb                swap(row_id, view_context)
lib/spree/email_block_registry.rb           view_path → blocks → partial + locals
app/controllers/spree/admin/email_templates_controller.rb
db/email_templates/<view_path>/<name>/{ template.json, thumb.png }
script/check_email_templates                a gate, like everything risky here
```

Samples ship as files and are seeded on first use. No HTML is committed beside a sample: it is derived, and a committed copy goes stale.

---

6. Two things the resolver has to handle that are not obvious [probed]

It is consulted for layouts and for **every partial**, twice each. It must return nothing cheaply and never answer a partial lookup by accident.

`full_document_templates.rb` in this app decides whether to apply Spree's layout using `File.exist?`. A resolver template has no file, so that check always says false and the layout is always applied, wrapping a full `<html>` export inside another `<html>` — which Gmail and Outlook strip, discarding the design.

---

7. Not decided yet

1. **Is Nokogiri the right tool for the swap.** Unmeasured, and it cuts both ways: libxml2 lowercases tags, rewrites the content-type meta, and does not preserve namespaced elements as authored, which is the exact

shape of the VML buttons the email pipeline emits for Outlook. Measure a real exported email through a parse-and-serialise round trip before committing to it. A string scan over the markers may be both safer and smaller.

2. **Preview and test send.** Not designed. A merchant who cannot see the email before it goes out will send a broken one, and this project has already mailed raw `**` and an unfilled interpolation to real customers.
3. **Rollback by a merchant, at 2am, without a developer.** Deleting the row restores the stock email, but nothing in the admin says so yet.
4. **Other locales.** The builder edits one locale's rendered copy. A shop running two gets one template unless we say otherwise.
5. **Block CSS.** `confirm_email` renders its styles through `content_for :head`. A block dropped into a builder page has no such channel.
6. **`emails:smoke` must cover saved templates**, and must stop passing a single-part message.